```
0000 separate (IDL.SEM_PHASE)¶¶¶¬¬¬¬¬--------¬package body¬¬¬VIS_UTIL¶is¬¬¬¬¬----------¬¶¶  use EXP_TYPE;¶  use DEF_UTIL;¶  use MAKE_NOD;¶  use REQ_UTIL;¶¶¬¬
     ¬----------¬¬procedure¬¬INIT_PARAM_CURSOR¶¬¬¬----------¬¶¬¬¬¬¬¬( CURSOR¬    :out PARAM_CURSOR_TYPE;¶¬¬¬¬¬¬  PARAM_LIST :SEQ_TYPE¶¬¬¬¬¬¬)¶
      is¶  begin¶    CURSOR.PARAM_LIST := PARAM_LIST;¶    CURSOR.ID_LIST¬  := ( TREE_NIL, TREE_NIL );¶¶¬----------¬¶  end¬INIT_PARAM_CURSOR;¶¬------
     ----------¬¶¶¶¬¬¬----------¬¬procedure¬¬ADVANCE_PARAM_CURSOR¶¬¬¬----------¬¶¬¬¬¬¬¬ ( CURSOR¬:in out PARAM_CURSOR_TYPE )¶  is¶  begin
     ¶    if IS_EMPTY( CURSOR.ID_LIST ) then¶      if IS_EMPTY( CURSOR.PARAM_LIST ) then¶        CURSOR.ID := TREE_VOID;¶        return;¶      else¶
      PARAM_CURSOR.PARAM_LIST, CURSOR.PARAM );¶        if CURSOR.PARAM.TY = DN_NULL_COMP_DECL then¶¬CURSOR.ID¬:= TREE_VOID;¶¬return;¶      end if;¶
      CURSOR.ID_LIST := LIST( D( AS_SOURCE_NAME_S, CURSOR.PARAM ) );¶      end¬if;¶    end if;¶    POP( CURSOR.ID_LIST, CURSOR.ID );¶¶¬----------
     ¶  end¬ADVANCE_PARAM_CURSOR;¶¬----------¬¶¶¶  procedure REDUCE_NAME_TYPES¬(DEFSET :¬in out DEFSET_TYPE;¬TYPESET :¬out TYPESET_TYPE);¶¶  proc
     edure FIND_SELECTED_DEFS (NAME_TYPESET : in¬out TYPESET_TYPE; DESIGNATOR : TREE; DEFSET : out¬DEFSET_TYPE);¶¶  procedure DEBUG_PRINT_DEF (DEF : TREE)
     is¶    HEADER      : TREE¬      :=¬D (XD_HEADER, DEF);¶    REGION      : TREE¬      :=¬D (XD_REGION_DEF, DEF);¶    PARAM_CURSOR  : PARAM_CURS
0010 OR_TYPE;¶    PAREN_OR_COMMA : String (1 .. 1) :=¬"(";¶  begin¶¶    Put ("    ");¶    Put (NODE_REP (DEF));¶    Put (" ");¶    Put (NODE_REP (D (XD_SOU
     RCE_NAME, DEF)));¶    Put (" IN ");¶    Put (NODE_REP (REGION));¶    Put (INTEGER'IMAGE (DI (XD_LEX_LEVEL, REGION)));¶    Put (" ");¶    Put_Line (BOO
     LEAN'IMAGE (DB (XD_IS_USED, REGION)));¶    if HEADER.TY in¬CLASS_SUBP_ENTRY_HEADER then¶      Put¬(ASCII.HT¬& "(");¶      INIT_PARAM_CURSOR (PARAM_CUR
     SOR, LIST (D (AS_PARAM_S,¬HEADER)));¶      loop¶        Put (PAREN_OR_COMMA);¶        PAREN_OR_COMMA := ",";¶        ADVANCE_PARAM_CURSOR (PARAM_CURSO
     R);¶        exit when PARAM_CURSOR.ID = TREE_VOID;¶        Put (NODE_REP (GET_BASE_TYPE (D¬(SM_OBJ_TYPE, PARAM_CURSOR.ID))));¶      end¬loop;¶      Pu
     t¬(")");¶    if HEADER.TY = DN_FUNCTION_SPEC then¶        Put ("->");¶        Put (NODE_REP (GET_BASE_TYPE (D¬(AS_NAME,¬HEADER))));¶      end¬if;¶
      New_Line;¶    end if;¶  end DEBUG_PRINT_DEF;¶¶  procedure FIND_VISIBILITY (EXP : TREE; DEFSET :¬out DEFSET_TYPE) is¶¬      -- FOR EXP, A¬USED_OBJE
     CT_ID OR A¬SELECTED,¬RETURN SET¶¬      -- OF DEF NODES FOR VISIBLE DECLARATIONS OF¬THE USED_OBJECT_ID¶¬      -- OR OF THE DESIGNATOR¬OF THE SELECTED.
      (NOTE: BUILTIN¶¬      -- OPERATIONS¬ARE NOT CONSIDERED.)¶¶  begin¶    case EXP.TY is¶      when CLASS_DESIGNATOR =>¶        FIND_DIRECT_VISIBILITY (E
     XP, DEFSET);¶      when DN_SELECTED =>¶        FIND_SELECTED_VISIBILITY (EXP, DEFSET);¶      when others =>¶        Put_Line ("!! INVALID ARGUMENT FOR
0020  FIND_VISIBILITY");¶        raise Program_Error;¶    end case;¶  end FIND_VISIBILITY;¶¶¶¬¬----------¬  procedure¬FIND_DIRECT_VISIBILITY¬(
     ID :TREE; DEFSET :out DEFSET_TYPE )¶¬¬----------¬  is¶¬      -- RETURNS SET OF DIRECTLY-VISIBLE DEF'S FOR USED_OBJECT_ID¶¶    NEST_UNIQUE,
      USED_UNIQUE¬: TREE¬ := TREE_VOID;¶    NEST_OVLOAD, USED_OVLOAD¬: SEQ_TYPE := (TREE_NIL, TREE_NIL);¶    NEST_UNIQUE_LEVEL¬: Natural¬ := 0;¶    USED_IS
     _OK¬¬: BOOLEAN¬ := True;¶¶    DEFLIST    : SEQ_TYPE := LIST (D (LX_SYMREP, ID));¶    DEF¬    : TREE;¶    LEVEL¬    : INTEGER;¶    REGION_DEF : TREE;
     ¶¶    DEFLIST_1, DEFLIST_2 : SEQ_TYPE;¶    DEF_1, DEF_2¬     : TREE;¶¶    NEW_DEFSET : DEFSET_TYPE := EMPTY_DEFSET;¶  begin¶¶¬      -- FOR EACH DEF FO
     R THIS NAME¶    while¬not IS_EMPTY (DEFLIST) loop¶      POP¬(DEFLIST,¬DEF);¶¶¬¬    -- IF¬IT IS POTENTIALLY STILL VALID¬(I.E., REGION IS DEFINED)¶
      REGION_DEF :=¬D (XD_REGION_DEF, DEF);¶      if REGION_DEF¬/= TREE_VOID then¶¶¬¬¬  -- IF IT IS DEFINED IN CURRENT OR ENCLOSING REGION¶        LEVEL :=
      DI¬(XD_LEX_LEVEL, REGION_DEF);¶      if LEVEL > 0 then¶¶¬¬¬¬-- IF IT IS OVERLOADABLE¶¬if IS_OVERLOADABLE_HEADER (D (XD_HEADER, DEF)) then¶¶¬¬¬¬
         -- ADD TO LIST OF OVERLOADABLE DEFS¶¬  NEST_OVLOAD := APPEND (NEST_OVLOAD, DEF);¶¶¬¬¬¬        -- ELSE IF IT IS NOT OVERLOADABLE¶¬¬¬¬        -- ...
      AND EITHER HIDES PRIOR NESTED NON-OVERLOADABLE¶¬¬¬¬        -- ...      OR ERROR AT SAME LEVEL AS PRIOR NON-OVERLOADABLE¶¬elsif (LEVEL > NEST_UNIQUE_LE
0030 VEL) or else (LEVEL = NEST_UNIQUE_LEVEL and then D¬(XD_HEADER, DEF) = TREE_FALSE) then¶¶¬¬¬¬        -- REMEMBER¬THIS DEF AS NON-OVERLOADABLE NESTED¶¬
      NEST_UNIQUE_LEVEL¬:= LEVEL;¶¬  NEST_UNIQUE¬:= DEF;¶¶¬¬¬¬        -- DISALLOW¬USED DEFS¶¬  USED_IS_OK := False;¶¬end if;¶¶¬¬¬¬-- ELSE IF IT USED DEFS A
     RE NOT KNOWN TO BE DISALLOWED¶¬¬¬¬-- ... AND THE REGION ENCLOSING THIS DEF HAS A USE CLAUSE¶¬¬¬¬-- ... AND THIS DEF¬IS FROM THE VISIBLE¬PART¶        e
     lsif USED_IS_OK and then DB (XD_IS_USED,¬REGION_DEF) and then DB (XD_IS_IN_SPEC,¬DEF) then¶¶¬¬¬¬-- IF THIS DEF IS OVERLOADABLE AND NOT ENTRY¶¬if IS_OV
     ERLOADABLE_HEADER (D (XD_HEADER, DEF)) and then D (XD_SOURCE_NAME, DEF).TY /= DN_ENTRY then¶¶¬¬¬¬        -- ADD TO LIST OF OVERLOADABLE USED DEFS¶¬  U
     SED_OVLOAD := APPEND (USED_OVLOAD, DEF);¶¶¬¬¬¬        -- ELSE IF THIS IS FIRST NON-OVERLOADABLE¬USED ENTRY¶¬¬¬¬        -- ... OR THERE WAS AN ERROR IN
     ¬ITS DECLARATION¶¬elsif USED_UNIQUE =¬TREE_VOID¬or else D¬(XD_HEADER, DEF) = TREE_FALSE¬then¶¶¬¬¬¬        -- SAVE THIS DEF AS NON-OVERLOADABLE NESTED
     DEF¶¬  USED_UNIQUE := DEF;¶¶¬¬¬¬        -- ELSE -- SINCE THIS¬IS A DUPLICATE NON-OVERLOADABLE USED¶¬else¶¶¬¬¬¬        -- DISALLOW¬USED DEFS¶¬  USED_IS
     _OK := False;¶¬end if;¶        end if;¶      end¬if;¶    end loop;¶¶¬      -- IF THERE ARE BOTH NON-OVERLOADABLE AND OVERLOADABLE NESTED DEFS¶    if N
     EST_UNIQUE /= TREE_VOID and then not IS_EMPTY (NEST_OVLOAD) then¶¶¬¬      -- DISCARD HIDDEN NESTED DEFS¶      declare¶        TEMP_OVLOAD¬: TREE;¶
0040   NEW_DEFLIST¬: SEQ_TYPE := (TREE_NIL, TREE_NIL);¶      begin¶        while not IS_EMPTY (NEST_OVLOAD) loop¶¬POP (NEST_OVLOAD, TEMP_OVLOAD);¶¬if DI (X
     D_LEX_LEVEL, D (XD_REGION_DEF, TEMP_OVLOAD)) > NEST_UNIQUE_LEVEL then¶¬  NEW_DEFLIST := APPEND (NEW_DEFLIST, TEMP_OVLOAD);¶¬  NEST_UNIQUE := TREE_VOID
     ;¶¬end if;¶      end loop;¶      NEST_OVLOAD¬:= NEW_DEFLIST;¶      end;¶¶¬¬    -- DISALLOW USED DEFS¶      USED_IS_OK :=¬False;¶    end if;¶¶¬
      -- IF THERE IS A VISIBLE NON-OVERLOADABLE NESTED DEF¶    if NEST_UNIQUE /= TREE_VOID then¶¶      declare¶        HEADER¬¬: TREE¬¬:= D( XD_HEADER, NE
     ST_UNIQUE );¶        HEADER_KIND¬¬: NODE_NAME;¶      begin¶¶        begin¶¬if  HEADER.PT = HI¬then HEADER_KIND :=¬HEADER.NOTY;¶¬elsif  HEADER.PT /=¬S
      then HEADER_KIND¬:= HEADER.TY;¶¬else raise PROGRAM_ERROR;¶¬end if;¶        exception¶¬when PROGRAM_ERROR =>¶¬  PUT( "IDL-SEM_PHASE-VISUTIL-FIND_DIREC
     T_VISIBILITY : HEADER TYPE ERROR " );¶¬  PUT( "(PT=>" & VPTR_TYPE'IMAGE( HEADER.PT ) & ")" );¶¬  raise;¶        end;¶¬¬¬  -- IF IT IS NOT YET FULLY DE
     CLARED OR¬IN ERROR¶        if HEADER_KIND in CLASS_BOOLEAN¬then¶¶¬¬¬¬-- EMPTY DEFSET IS TO BE RETURNED¶¬¬¬¬-- PUT OUT CORRECT ERROR OR WARNING MESSAGE
     ¶¬if HEADER_KIND = DN_FALSE then¶¬  WARNING( D( LX_SRCPOS, ID ), "PRIOR ERROR IN DECLARATION - " & PRINT_NAME( D(¬LX_SYMREP, ID ) ) );¶¬else¶¬  ERROR(
      D( LX_SRCPOS, ID ), "NAME NOT YET VISIBLE - " & PRINT_NAME( D( LX_SYMREP, ID ) )¬);¶¬end if;¶¶¬¬¬¬-- ELSE -- SINCE IT¬IS FULLY DECLARED AND NOT IN ER

0050 ROR¶        else¶¶¬¬¬¬-- THIS IS THE CORRECT DEF¶¬ADD_TO_DEFSET( NEW_DEFSET, NEST_UNIQUE );¶        end if;¶¶¬¬¬  -- RETURN NEW DEFSET¶        DEFSET
     := NEW_DEFSET;¶        return;¶¶      end;¶    end if;¶¶¬      -- HERE, EITHER THERE ARE NO NESTED DEFS OR¬ALL ARE OVERLOADABLE¶¶¬      -- IF USED DEF
     S HAVE BEEN DISALLOWED¶¬      -- ... (BECAUSE NON-OVERLOADED NEST OR BECAUSE MULTIPLE NON-OVERLOADED)¶    if not USED_IS_OK then¶¶¬¬      -- CLEAR USED
     DEFS¶      USED_UNIQUE := TREE_VOID;¶      USED_OVLOAD := (TREE_NIL, TREE_NIL);¶    end if;¶¶¬      -- IF THERE IS A NON-OVERLOADABLE¬USED DEF¶    if
     USED_UNIQUE /= TREE_VOID then¶¶¬¬    -- IF¬IT IS FROM A DECLARATION WHICH WAS IN ERROR¶      if D (XD_HEADER, USED_UNIQUE) = TREE_FALSE then¶¶¬¬¬  --
     PRINT WARNING¶        WARNING (D (LX_SRCPOS, ID), "PRIOR ERROR IN (USED) DECLARATION - " & PRINT_NAME (D (LX_SYMREP, ID)));¶¶¬¬¬  -- RETURN EMPTY DEFS
     ET¶        DEFSET := EMPTY_DEFSET;¶        return;¶¶¬¬  -- ELSE¬IF THERE ARE NO OVERLOADABLE DEFS¶      elsif IS_EMPTY (NEST_OVLOAD) and then IS_EMPT
     Y (USED_OVLOAD) then¶¶¬¬¬  -- RETURN THE (UNIQUE) NON-OVERLOADABLE USED DEF¶        ADD_TO_DEFSET (NEW_DEFSET, USED_UNIQUE);¶        DEFSET := NEW_DEF
     SET;¶        return;¶¶¬¬¬  -- ELSE¬-- SINCE (1) OVERLOADABLE AND¬(2) NON-OVERLOADABLE USED DEFS¶      else¶¶¬¬¬  -- DISCARD ALL USED DEFS¶        USED
     _UNIQUE¬:= TREE_VOID;¶        USED_OVLOAD¬:= (TREE_NIL, TREE_NIL);¶      end¬if;¶    end if;¶¶¬      -- FIND NESTED DEFS WHICH ARE NOT¬HIDDEN¶    DEFL
0060 IST_1 := NEST_OVLOAD;¶    while¬not IS_EMPTY (DEFLIST_1) loop¶      POP¬(DEFLIST_1, DEF_1);¶¶      DEFLIST_2 := NEST_OVLOAD;¶      while not IS_EMPTY
     (DEFLIST_2) loop¶        DEF_2 := HEAD (DEFLIST_2);¶        if DEF_2 /=¬DEF_1 and¬then ARE_HOMOGRAPH_HEADERS (D¬(XD_HEADER, DEF_1),¬D (XD_HEADER, DEF_
     2)) then¶¬if DI (XD_LEX_LEVEL, D (XD_REGION_DEF, DEF_1)) > DI (XD_LEX_LEVEL, D (XD_REGION_DEF, DEF_2)) then¶¬  null;¶¬elsif DI (XD_LEX_LEVEL, D (XD_RE
     GION_DEF, DEF_1))¬< DI (XD_LEX_LEVEL,¬D (XD_REGION_DEF, DEF_2)) then¶¬¬¬¬        -- HIDDEN BY DEF_2¶¬  exit;¶¬elsif D (XD_SOURCE_NAME, DEF_1).TY = DN_
     BLTN_OPERATOR_ID or¬else D (XD_SOURCE_NAME, DEF_1).TY in CLASS_ENUM_LITERAL then¶¬¬¬¬        -- HIDDEN BY DEF_2¶¬  exit;¶¬elsif D (XD_SOURCE_NAME, DEF
     _2).TY = DN_BLTN_OPERATOR_ID or¬else D (XD_SOURCE_NAME, DEF_2).TY in CLASS_ENUM_LITERAL then¶¬  null;¶¬elsif D (XD_SOURCE_NAME, DEF_1).TY in CLASS_SUB
     PROG_NAME¶¬ and then¶¬ (D (SM_UNIT_DESC, D (XD_SOURCE_NAME, DEF_1)).TY = DN_DERIVED_SUBPROG¶¬  or else¬(D (SM_UNIT_DESC, D¬(XD_SOURCE_NAME, DEF_1)).TY
     =¬DN_IMPLICIT_NOT_EQ and then D¬(SM_UNIT_DESC, D (SM_EQUAL, D¬(SM_UNIT_DESC, D (XD_SOURCE_NAME, DEF_1)))).TY = DN_DERIVED_SUBPROG))¶¬then¶¬¬¬¬
      -- HIDDEN BY DEF_2¶¬  exit;¶¬elsif D (XD_SOURCE_NAME, DEF_2).TY in CLASS_SUBPROG_NAME¶¬ and then¶¬ (D (SM_UNIT_DESC, D (XD_SOURCE_NAME, DEF_2)).TY =
     DN_DERIVED_SUBPROG¶¬  or else¬(D (SM_UNIT_DESC, D¬(XD_SOURCE_NAME, DEF_2)).TY =¬DN_IMPLICIT_NOT_EQ and then D¬(SM_UNIT_DESC, D (SM_EQUAL, D¬(SM_UNIT_D
0070 ESC, D (XD_SOURCE_NAME, DEF_1)))).TY = DN_DERIVED_SUBPROG))¶¬then¶¬  null;¶¬else¶¬¬¬¬        -- HIDDEN BY DEF_2¶¬  exit;¶¬end if;¶        end if;¶
       DEFLIST_2 := TAIL (DEFLIST_2);¶      end¬loop;¶      if IS_EMPTY (DEFLIST_2)¬then¶        ADD_TO_DEFSET (NEW_DEFSET, DEF_1);¶      end¬if;¶    end
      loop;¶¶¬      -- FIND USED DEFS WHICH¬ARE NOT HIDDEN¶    DEFLIST_1 := USED_OVLOAD;¶    while¬not IS_EMPTY (DEFLIST_1) loop¶      POP¬(DEFLIST_1, DEF_
     1);¶¶¬¬    -- CHECK FOR USED DEFS HIDDEN BY NESTED DEFS¶      DEFLIST_2 := NEST_OVLOAD;¶      while not IS_EMPTY (DEFLIST_2) loop¶        DEF_2 := HEA
     D (DEFLIST_2);¶        if ARE_HOMOGRAPH_HEADERS (D (XD_HEADER, DEF_1), D (XD_HEADER,¬DEF_2)) then¶¬¬¬¬-- HIDDEN¬BY DEF_2¶¬exit;¶        end if;¶
       DEFLIST_2 := TAIL (DEFLIST_2);¶¶      end¬loop;¶¶¬¬      -- IF¬NOT HIDDEN BY NESTED DEF¶      if IS_EMPTY (DEFLIST_2)¬then¶¶¬¬¬  -- CHECK IF HIDDEN BY
     ANOTHER USED DEF¶        DEFLIST_2 := USED_OVLOAD;¶        while not IS_EMPTY (DEFLIST_2) loop¶¬DEF_2 := HEAD (DEFLIST_2);¶¬if DEF_2 /= DEF_1 and the
     n ARE_HOMOGRAPH_HEADERS (D (XD_HEADER, DEF_1), D (XD_HEADER, DEF_2)) then¶¶¬  if D (XD_REGION_DEF, DEF_1)¬/= D (XD_REGION_DEF, DEF_2) then¶¬¬¬¬
     -- BOTH ARE MADE VISIBLE (BUT WILL BE AMBIGUOUS)¶¬    null;¶¬  elsif D¬(XD_SOURCE_NAME, DEF_1).TY = DN_BLTN_OPERATOR_ID or else D (XD_SOURCE_NAME, DEF
     _1).TY in CLASS_ENUM_LITERAL then¶¬¬¬¬        -- HIDDEN BY DEF_2¶¬    exit;¶¬  elsif D¬(XD_SOURCE_NAME, DEF_2).TY = DN_BLTN_OPERATOR_ID or else D (XD_S
0080 OURCE_NAME, DEF_2).TY in CLASS_ENUM_LITERAL then¶¬    null;¶¬  elsif D¬(XD_SOURCE_NAME, DEF_1).TY in¬CLASS_SUBPROG_NAME¶¬   and then¶¬   (D (SM_UNIT_D
     ESC, D (XD_SOURCE_NAME,¬DEF_1)).TY = DN_DERIVED_SUBPROG¶¬    or else (D (SM_UNIT_DESC,¬D (XD_SOURCE_NAME, DEF_1)).TY¬= DN_IMPLICIT_NOT_EQ and then¬D (
     SM_UNIT_DESC, D (SM_EQUAL,¬D (SM_UNIT_DESC, D (XD_SOURCE_NAME, DEF_1)))).TY = DN_DERIVED_SUBPROG))¶¬  then¶¬¬¬¬        -- HIDDEN BY DEF_2¶¬    exit;¶¬
      else D¬(XD_SOURCE_NAME, DEF_2).TY in¬CLASS_SUBPROG_NAME¶¬   and then¶¬   (D (SM_UNIT_DESC, D (XD_SOURCE_NAME,¬DEF_2)).TY = DN_DERIVED_SUBPROG¶¬    o
     r else (D (SM_UNIT_DESC,¬D (XD_SOURCE_NAME, DEF_2)).TY¬= DN_IMPLICIT_NOT_EQ and then¬D (SM_UNIT_DESC, D (SM_EQUAL,¬D (SM_UNIT_DESC, D (XD_SOURCE_NAME,
      DEF_1)))).TY = DN_DERIVED_SUBPROG))¶¬  then¶¬    null;¶¬  else¶¬¬¬¬        -- HIDDEN BY DEF_2¶¬    exit;¶¬  end if;¶¬end if;¶¬DEFLIST_2¬:= TAIL (DEFLI
     ST_2);¶      end loop;¶      end¬if;¶      if IS_EMPTY (DEFLIST_2)¬then¶        ADD_TO_DEFSET (NEW_DEFSET, DEF_1);¶      end¬if;¶    end loop;¶¶
     if IS_EMPTY (NEW_DEFSET) then¶      ERROR (D (LX_SRCPOS, ID), "NO DIRECTLY VISIBLE DECLARATION - " & PRINT_NAME (D (LX_SYMREP, ID)));¶    end if;¶
     DEFSET := NEW_DEFSET;¶  end FIND_DIRECT_VISIBILITY;¶¶        ----------¬----------¬----------¬----------
     ----------¬¶¶  procedure FIND_SELECTED_VISIBILITY (SELECTED : TREE; DEFSET : out DEFSET_TYPE) is¶¬      -- GIVEN
0090 A SELECTED NODE, FIND ALL VISIBLE DEF'S FOR THE DESIGNATOR¶¶    NAME¬      : TREE := D (AS_NAME, SELECTED);¶    DESIGNATOR : TREE := D (AS_DESIGNATOR,
     SELECTED);¶¶    NEW_DEFSET : DEFSET_TYPE := EMPTY_DEFSET;¶¶    NAME_DEFSET    : DEFSET_TYPE  := EMPTY_DEFSET;¶    NAME_DEFINTERP : DEFINTERP_TYPE;¶
      NAME_DEF      : TREE¬     := TREE_VOID;¶    NAME_TYPESET   : TYPESET_TYPE := EMPTY_TYPESET;¶¶    TEMP_LIST : SEQ_TYPE;¶    TEMP¬    : TREE;¶  begin¶¬
          -- IF DESIGNATOR IS A STRING, MAKE IT A USED_OP¶    if DESIGNATOR.TY = DN_STRING_LITERAL then¶      DESIGNATOR :=¬MAKE_USED_OP_FROM_STRING (DESI
     GNATOR);¶      D (AS_DESIGNATOR, SELECTED, DESIGNATOR);¶    end if;¶¶¬      -- ACCORDING TO THE KIND OF PREFIX¶    case CLASS_EXP'(NAME.TY) is¶¶
     when DN_USED_OBJECT_ID =>¶¬¬¬  -- FOR USED_OBJECT_ID, FIND¬DIRECT VISIBILITY¶        FIND_DIRECT_VISIBILITY (NAME, NAME_DEFSET);¶¶      when DN_STRING
     _LITERAL =>¶¬¬¬  -- FOR STRING, MAKE IT A USED_OP AND FIND DIRECT VISIBILITY¶        NAME := MAKE_USED_OP_FROM_STRING (NAME);¶        D¬(AS_NAME,¬SELE
     CTED,¬NAME);¶¶        FIND_DIRECT_VISIBILITY (NAME, NAME_DEFSET);¶¶      when DN_SELECTED =>¶¬¬¬  -- FOR SELECTED, FIND SELECTED VISIBILITY¶        FI
     ND_SELECTED_VISIBILITY (NAME,¬NAME_DEFSET);¶¶      when others =>¶¬¬¬  -- OTHERWISE, MUST BE EXPRESSION; FIND POSSIBLE¬TYPES¶        EVAL_EXP_TYPES (N
     AME,¬NAME_TYPESET);¶    end case;¶¶¬      -- IF WE FOUND SOME NAME DEF'S¶    if not IS_EMPTY¬(NAME_DEFSET) then¶¶¬¬    -- IF¬THERE IS AN ENCLOSING REG

0100 ION¶      NAME_DEF := GET_ENCLOSING_DEF (NAME, NAME_DEFSET);¶      if NAME_DEF /= TREE_VOID then¶¶¬¬¬  -- IT'S¬THE ONLY INTERPRETATION OF THE NAME¶¬¬¬
       -- LOOK¬FOR ENTITIES IMMEDIATELY WITHIN THE ENCLOSING REGION¶¬¬¬  -- (NOTE.  RM 4.1.3/10 HAS PREFERENCE¬RULE ONLY¬FOR¶¬¬¬  --  ...¬ENCLOSING¬SUBPROG
     RAM OR ACCEPT STATEMENT; HOWEVER¶¬¬¬  --  ...¬IF, E.G.,¬ENCLOSING¬PACKAGE, ONLY ONE IS VISIBLE ANYWAY)¶        TEMP_LIST := LIST (D (LX_SYMREP, DESIGN
     ATOR));¶        while not IS_EMPTY (TEMP_LIST) loop¶¬POP (TEMP_LIST, TEMP);¶¬if D (XD_REGION_DEF, TEMP) = NAME_DEF then¶¬  ADD_TO_DEFSET (NEW_DEFSET,
     TEMP);¶¬end if;¶        end loop;¶¶¬¬  -- ELSE¬IF PREFIX¬IS A PACKAGE NAME¶      elsif GET_THE_ID (NAME_DEFSET).TY¬= DN_PACKAGE_ID then¶¶¬¬¬  -- IT'S
     ¬THE ONLY INTERPRETATION OF THE NAME¶¬¬¬  -- CHECK FOR RENAMING; USE ORIGINAL PACKAGE¶        NAME_DEFINTERP := HEAD (NAME_DEFSET);¶        NAME_DEF¬
       := GET_DEF (NAME_DEFINTERP);¶        if D (SM_UNIT_DESC, D¬(XD_SOURCE_NAME, NAME_DEF)).TY = DN_RENAMES_UNIT then¶¬NAME_DEF := GET_DEF_FOR_ID (GET_BA
     SE_PACKAGE (D (XD_SOURCE_NAME, NAME_DEF)));¶        end if;¶¶¬¬¬  -- LOOK¬FOR ENTITIES DEFINED IMMEDIATELY WITHIN¬SPECIFICATION¶        TEMP_LIST := L
     IST (D (LX_SYMREP, DESIGNATOR));¶        while not IS_EMPTY (TEMP_LIST) loop¶¬POP (TEMP_LIST, TEMP);¶¬if D (XD_REGION_DEF, TEMP) = NAME_DEF and then D
     B¬(XD_IS_IN_SPEC, TEMP) then¶¬  ADD_TO_DEFSET (NEW_DEFSET, TEMP);¶¬end if;¶        end loop;¶      end¬if;¶    end if;¶¶¬      -- IF IT IS AN ENCLOSING
0110 ¬NAME¶    if NAME_DEF /= TREE_VOID then¶¶¬¬    -- MAKE THE PREFIX A USED_NAME_ID IF IT IS AN¬IDENTIFIER¶¬¬    -- AND STORE THE DEFINITION¶      if NAM
     E.TY = DN_SELECTED then¶        if D (AS_DESIGNATOR, NAME).TY =¬DN_USED_OBJECT_ID then¶¬D (AS_DESIGNATOR, NAME, MAKE_USED_NAME_ID_FROM_OBJECT (D (AS_D
     ESIGNATOR, NAME)));¶      end if;¶        D¬(SM_DEFN,¬D (AS_DESIGNATOR, NAME), D (XD_SOURCE_NAME, NAME_DEF));¶      else¶      if NAME.TY = DN_USE
     D_OBJECT_ID then¶¬NAME := MAKE_USED_NAME_ID_FROM_OBJECT (NAME);¶¬D (AS_NAME, SELECTED, NAME);¶        end if;¶        D¬(SM_DEFN,¬NAME, D (XD_SOURCE_N
     AME, NAME_DEF));¶      end¬if;¶¶¬¬    -- DISCARD HIDDEN IMPLICIT SUBPROGRAMS¶      declare¶        OLD_DEFSET¬   : DEFSET_TYPE :=¬NEW_DEFSET;¶
     OLD_DEFINTERP  : DEFINTERP_TYPE;¶        OLD_ID¬    : TREE;¶        TEMP_DEFSET¬   : DEFSET_TYPE;¶        TEMP_DEFINTERP : DEFINTERP_TYPE;¶        TEMP
     _ID¬   : TREE;¶        NEW_NEW_DEFSET : DEFSET_TYPE :=¬EMPTY_DEFSET;¶      begin¶        while not IS_EMPTY (OLD_DEFSET)¬loop¶¬POP (OLD_DEFSET, OLD_DE
     FINTERP);¶¬OLD_ID :=¬D (XD_SOURCE_NAME, GET_DEF (OLD_DEFINTERP));¶¬if OLD_ID.TY = DN_BLTN_OPERATOR_ID or else OLD_ID.TY in CLASS_ENUM_LITERAL or else
     (OLD_ID.TY in CLASS_SUBPROG_NAME and¬then D (SM_UNIT_DESC, OLD_ID).TY = DN_INSTANTIATION) then¶¬  TEMP_DEFSET := NEW_DEFSET;¶¬  while not IS_EMPTY (TE
     MP_DEFSET) loop¶¬    TEMP_DEFINTERP := HEAD (TEMP_DEFSET);¶¬    if TEMP_DEFINTERP /= OLD_DEFINTERP and then ARE_HOMOGRAPH_HEADERS¬(D (XD_HEADER, GET_D
0120 EF (OLD_DEFINTERP)), D (XD_HEADER, GET_DEF (TEMP_DEFINTERP))) then¶¬      if OLD_ID.TY = DN_BLTN_OPERATOR_ID or else OLD_ID.TY in CLASS_ENUM_LITERAL t
     hen¶¬        exit;¶¬      else¶¬        TEMP_ID := D (XD_SOURCE_NAME, GET_DEF (TEMP_DEFINTERP));¶¬        if TEMP_ID.TY /= DN_BLTN_OPERATOR_ID and the
```

```
n TEMP_ID.TY not in CLASS_ENUM_LITERAL then¶¬¬exit;¶¬        end if;¶¬      end¬if;¶¬    end if;¶¬    POP (TEMP_DEFSET, TEMP_DEFINTERP);¶¬  end loop;¶
¬  if IS_EMPTY (TEMP_DEFSET) then¶¬    ADD_TO_DEFSET (NEW_NEW_DEFSET, OLD_DEFINTERP);¶¬  end if;¶¬else¶¬  ADD_TO_DEFSET (NEW_NEW_DEFSET, OLD_DEFINTERP
);¶¬end if;¶        end loop;¶        NEW_DEFSET := NEW_NEW_DEFSET;¶    end;¶¶¬¬    -- ELSE IF IT IS A DEFINED NAME OR A SELECTED, IT MUST BE AN EXP
RESSION¶    elsif¬not IS_EMPTY (NAME_DEFSET) then¶      REDUCE_NAME_TYPES (NAME_DEFSET, NAME_TYPESET);¶      STASH_DEFSET (NAME, NAME_DEFSET);¶    end
 if;¶¶¬      -- IF EXPRESSION, ONLY CONSIDER TYPES WHICH¬HAVE SELECTED COMPONENTS¶    if not IS_EMPTY¬(NAME_TYPESET) then¶      FIND_SELECTED_DEFS (NA
ME_TYPESET,¬DESIGNATOR, NEW_DEFSET);¶    end if;¶    if NAME.TY /= DN_USED_NAME_ID then¶      STASH_TYPESET¬(NAME, NAME_TYPESET);¶    end if;¶¶¬
-- CHECK FOR NO DECLARATIONS FOUND¶    if IS_EMPTY (NEW_DEFSET) and then not (IS_EMPTY (NAME_DEFSET) and¬then IS_EMPTY (NAME_TYPESET))¬then¶      ERRO
R (D (LX_SRCPOS, DESIGNATOR),¬"NOT VISIBLE BY SELECTION - "¬& PRINT_NAME (D (LX_SYMREP, DESIGNATOR)));¶¶¬¬    -- CHECK FOR ERROR OR NOT-YET-VISIBLE DE
CLARATION¶    else¶      declare¶        TEMP_DEFSET¬¬: DEFSET_TYPE¬:= NEW_DEFSET;¶        TEMP_DEFINTERP¬: DEFINTERP_TYPE;¶      begin¶¶        while
  not IS_EMPTY( TEMP_DEFSET )  loop¶¬POP( TEMP_DEFSET, TEMP_DEFINTERP );¶¶¬¬¬¬-- IF IT IS NOT YET¬FULLY DECLARED OR IN ERROR¶¬declare¶¬  HEADER¬¬: TRE
E¬¬:= D( XD_HEADER, GET_DEF( TEMP_DEFINTERP ) );¶---¬  HEADER_KIND¬: NODE_NAME;¶¬begin¶---¬HEADER_KIND := D (XD_HEADER, GET_DEF (TEMP_DEFINTERP)).TY;¶¬
 if HEADER.PT = HI¬and then HEADER.NOTY in CLASS_BOOLEAN then¶¶¬¬¬¬        -- EMPTY DEFSET IS TO¬BE RETURNED¶¬    NEW_DEFSET := EMPTY_DEFSET;¶¶¬¬¬¬
     -- PUT OUT CORRECT ERROR OR WARNING MESSAGE¶¬    if HEADER.NOTY = DN_FALSE¬then¶¬      WARNING( D( LX_SRCPOS, DESIGNATOR¬), "PRIOR ERROR IN DECLA
RATION - " & PRINT_NAME( D( LX_SYMREP, DESIGNATOR ) ) );¶¬    else¶¬      ERROR( D( LX_SRCPOS, DESIGNATOR ), "NAME NOT YET VISIBLE - " & PRINT_NAME( D
( LX_SYMREP, DESIGNATOR ) )¬);¶¬    end if;¶¬  end if;¶¬end;¶        end loop;¶        end;¶    end if;¶¶¬      -- COPY RESULTS TO OUT ARGUMENT AND RETU
RN¶    DEFSET := NEW_DEFSET;¶  end FIND_SELECTED_VISIBILITY;¶¶        ------------------------------------------------------------------¶¶  function GE
_ENCLOSING_DEF (USED_NAME¬: TREE; DEFSET : DEFSET_TYPE)¬return TREE is¶¬      -- GETS INNERMOST ENCLOSING NAME IN DEFSET¶¬      -- DEFSET HAS¬NAMES DE
FINED IN ENCLOSING REGIONS FIRST¶¬      -- NOTE.  PARAMETER USED_NAME ONLY FOR ERROR MESSAGES¶¶    TEMP_DEFSET : DEFSET_TYPE¬:= DEFSET;¶    DEFINTERP
   : DEFINTERP_TYPE;¶    DEF¬      : TREE;¶¶    ENCLOSING_DEF¬: TREE¬:= TREE_VOID;¶    IS_MULTIPLE_DEF¬: BOOLEAN¬:= False;¶  begin¶¬      -- FOR EACH D
EF IN DEFSET¶    while¬not IS_EMPTY (TEMP_DEFSET) loop¶      POP¬(TEMP_DEFSET, DEFINTERP);¶      DEF¬:= GET_DEF (DEFINTERP);¶¶¬¬    -- STOP LOOKING¬IF
 NOT DEFINED IN ENCLOSING REGION¶      if DI (XD_LEX_LEVEL, D (XD_REGION_DEF, DEF)) = 0 then¶¬        exit;¶      end¬if;¶¶¬¬    -- IF¬IT IS AN ENCLOSI
NG REGION, HAVE FOUND ONE¶      if DI (XD_LEX_LEVEL, DEF) > 0 then¶¶¬¬¬  -- IF THIS IS THE¬FIRST ONE¬FOUND¶        if ENCLOSING_DEF = TREE_VOID then¶¶
¬¬¬¬-- THEN REMEMBER IT¶¬ENCLOSING_DEF := DEF;¶        else¶¬¬¬¬-- ELSE REMEMBER THAT ERROR OCCURRED¶¬IS_MULTIPLE_DEF := True;¶¶¬¬¬¬-- ALSO RETAIN MOS
T-DEEPLY-NESTED RESULT¶¬if DI (XD_LEX_LEVEL, DEF) > DI (XD_LEX_LEVEL, ENCLOSING_DEF) then¶¬  ENCLOSING_DEF := DEF;¶¬end if;¶        end if;¶      end¬
if;¶    end loop;¶¶¬      -- IF MULTIPLE DEFINITION WAS SEEN¶    if IS_MULTIPLE_DEF then¶¶¬¬    -- PUT OUT ERROR MESSAGE¶      ERROR (D (LX_SRCPOS, US
ED_NAME), "AMBIGUOUS ENCLOSING REGION");¶    end if;¶¶¬      -- RETURN MOST-DEEPLY-NESTED DEF,¬IF FOUND,¬OR VOID¶    return ENCLOSING_DEF;¶  end GET_E
NCLOSING_DEF;¶      --|------------------------------------------------------------------------------¶      --|¬      MAKE_USED_NAME_ID_F
ROM_OBJECT¶  function MAKE_USED_NAME_ID_FROM_OBJECT (USED_OBJECT_ID : TREE) return TREE is¶    SRC_POS : TREE := D (LX_SRCPOS, USED_OBJECT_ID);¶    SY

MREP  : TREE := D (LX_SYMREP, USED_OBJECT_ID);¶    DEFN¬  : TREE := D (SM_DEFN, USED_OBJECT_ID);¶  begin¶    return MAKE_USED_NAME_ID (LX_SRCPOS¬=> SR
C_POS, LX_SYMREP => SYMREP, SM_DEFN => DEFN);¶  end MAKE_USED_NAME_ID_FROM_OBJECT;¶¶        ------------------------------------------------------------
------¶¶  function MAKE_USED_OP_FROM_STRING (STRING_NODE : TREE) return TREE is¶¶    function MAKE_UPPER_CASE (A : String) return String is¶      A_WO
RK : String (1 .. A'LENGTH) := A;¶      MAGIC  : constant¬        := Character'POS ('A') - Character'POS ('A');¶    begin¶      for¬I in A_WORK'RANGE
 loop¶        if A_WORK (I) in 'A' .. 'Z' then¬A_WORK (I) := Character'VAL (Character'POS (A_WORK (I)) - MAGIC);¶        end if;¶      end¬loop;¶
 return A_WORK;¶    end MAKE_UPPER_CASE;¶¶  begin -- MAKE_USED_OP_FROM_STRING¶      return MAKE_USED_OP (LX_SRCPOS => D¬(LX_SRCPOS, STRING_NODE), LX_SYM
REP => STORE_SYM (MAKE_UPPER_CASE (PRINT_NAME (D (LX_SYMREP, STRING_NODE)))));¶  end MAKE_USED_OP_FROM_STRING;¶¶        ----------------------------
------------------------------¶¶  procedure REDUCE_NAME_TYPES¬(DEFSET :¬in out DEFSET_TYPE;¬TYPESET :¬out TYPESET_TYPE) is¶¬      -- REDUCES DEFSE
T TO NAMES WHICH HAVE A TYPE (ARE EXPRESSIONS)¶¬      -- (NOTE THAT¬FUNCTIONS¬REQUIRING¬PARAMETERS ARE DISCARDED HERE)¶¶    DEFINTERP : DEFINTERP_TYPE
;¶    DEF¬    : TREE;¶¶    NEW_DEFSET  : DEFSET_TYPE¬ := EMPTY_DEFSET;¶    NEW_TYPESET : TYPESET_TYPE := EMPTY_TYPESET;¶    TYPE_SPEC   : TREE;¶  begi
n¶    while¬not IS_EMPTY (DEFSET) loop¶      POP¬(DEFSET, DEFINTERP);¶      DEF¬      := GET_DEF (DEFINTERP);¶      TYPE_SPEC := EXPRESSION_TYPE_OF_DE
F (DEF);¶¶      if TYPE_SPEC /= TREE_VOID then¶        ADD_TO_DEFSET (NEW_DEFSET, DEFINTERP);¶        ADD_TO_TYPESET (NEW_TYPESET, TYPE_SPEC, GET_EXTR
AINFO (DEFINTERP));¶      end¬if;¶    end loop;¶¶    DEFSET  := NEW_DEFSET;¶    TYPESET := NEW_TYPESET;¶  end REDUCE_NAME_TYPES;¶¶        ------------
------------------------------------------------¶¶  function EXPRESSION_TYPE_OF_DEF (DEF : TREE) return TREE is¶¬      -- RETURNS BASE TYPE IF¬DEF
 REPRESENTS AN EXPRESSION¶¬      -- OTHERWISE RETURNS VOID¶¶    ID¬ : constant TREE :=¬D (XD_SOURCE_NAME, DEF);¶    HEADER : constant TREE :=¬D (XD_HE
ADER, DEF);¶  begin¶    if ID.TY = DN_NUMBER_ID then¶      if D (SM_OBJ_TYPE, ID).TY = DN_UNIVERSAL_REAL then¶        return MAKE¬(DN_ANY_REAL);¶
 else¶        return MAKE¬(DN_ANY_INTEGER);¶      end¬if;¶    elsif¬ID.TY in CLASS_OBJECT_NAME then¶      return GET_BASE_TYPE (D¬(SM_OBJ_TYPE, ID));¶
    elsif¬HEADER.TY¬= DN_FUNCTION_SPEC and then ALL_PARAMETERS_HAVE_DEFAULTS (HEADER) then¶      return GET_BASE_TYPE (D¬(AS_NAME,¬HEADER));¶    elsif
¬ID.TY in CLASS_TYPE_NAME and then GET_BASE_TYPE (ID).TY = DN_TASK_SPEC and then¬DI (XD_LEX_LEVEL, GET_DEF_FOR_ID (D (XD_SOURCE_NAME, GET_BASE_TYPE (I
D)))) > 0 then¶      return GET_BASE_TYPE (ID);¶    else¶      return TREE_VOID;¶    end if;¶  end EXPRESSION_TYPE_OF_DEF;¶¶        ------------------
-----------------------------------------¶¶  function ALL_PARAMETERS_HAVE_DEFAULTS¬(HEADER :¬TREE) return BOOLEAN is¶¬      -- GIVEN A SUBPROGRAM
 OR ENTRY HEADER, TEST¬IF ALL DECLARED¶¬      -- PARAMETERS¬HAVE A DEFAULT VALUE (OR THERE ARE NO PARAMETERS)¶¶    PARAM_LIST : SEQ_TYPE := LIST (D (A
S_PARAM_S,¬HEADER));¶    PARAM¬    : TREE;¶  begin¶¬      -- FOR EACH PARAMETER DECLARATION¶    while¬not IS_EMPTY (PARAM_LIST) loop¶      POP¬(PARAM
_LIST, PARAM);¶¶¬¬    -- IF¬IT DOES NOT HAVE A DEFAULT VALUE¶      if D (AS_EXP,¬PARAM) = TREE_VOID then¶¶¬¬¬  -- THEN¬ALL PARAMETERS DO NOT HAVE DEFA
ULTS; RETURN FALSE¶        return False;¶      end¬if;¶    end loop;¶¶¬      -- NO PARAMETERS FOUND WITHOUT DEFAULT; RETURN TRUE¶    return True;¶¶  e
nd ALL_PARAMETERS_HAVE_DEFAULTS;¶¶        ------------------------------------------------------------------¶¶        --- $$$$ TEMPORARY $$$$$$$$$$$$$$¶
  function IS_OVERLOADABLE_HEADER (HEADER : TREE)¬return BOOLEAN is¶  begin¶    return  (HEADER.PT = P  or  HEADER.PT = L)¶¬  and then¶¬  (HEADER.TY =
 DN_FUNCTION_SPEC or HEADER.TY = DN_PROCEDURE_SPEC or HEADER.TY =¬DN_ENTRY);¶¶  end¬IS_OVERLOADABLE_HEADER;¶¬----------------------¶¶¶¶  function CAST
_TREE (ARG : SEQ_TYPE) return TREE¬is¶  begin¶    return ARG.FIRST;¶  end CAST_TREE;¶¶  function CAST_SEQ_TYPE (ARG¬: TREE) return SEQ_TYPE is¶  begin
¶    return SINGLETON( ARG );¶  end CAST_SEQ_TYPE;¶¶¶¶¶¬¬¬---=====--¶¬function¬¬COPY_NODE¬¬( T : TREE ) return¬TREE¶¬¬¬--======--¶  is¶  begin¶    if T.
PT = HI or¬T.PT = S then¶      return T;¶    else¶      declare¶        LENGTH¬: ATTR_NBR¬:= DABS( 0, T ).NSIZ;¶        RESULT¬: TREE¬¬:= MAKE( T.TY,
 LENGTH );¶      begin¶        for I in 1 .. LENGTH loop¶¬DABS( I, RESULT, DABS( I, T )¬);¶        end loop;¶        return RESULT;¶      end;¶    end
 if;¶¶  end¬COPY_NODE;¶¬--======--¶¶¶¶        ------------------------------------------------------------------¶¶  procedure FIND_SELECTED_DEFS (NAME_TYP
ESET : in¬out TYPESET_TYPE; DESIGNATOR : TREE; DEFSET : out¬DEFSET_TYPE) is¶¬      -- GIVEN A LIST OF TYPES AND A DESIGNATOR, FIND THOSE¶¬      -- DEF
S FOR THE DESIGNATOR SUCH THAT SELECTED IS VALID EXPRESSION¶¶    DESIGNATOR_DEFLIST : constant SEQ_TYPE := LIST (D (LX_SYMREP, DESIGNATOR));¶    TEMP_
NAME_TYPESET  : TYPESET_TYPE¬   := NAME_TYPESET;¶    NAME_TYPEINTERP¬   : TYPEINTERP_TYPE;¶    NAME_STRUCT¬    : TREE;¶    NAME_TYPE_ID¬    : TREE;¶
NAME_DEF¬    : TREE;¶¶    TEMP_DEFLIST : SEQ_TYPE;¶    TEMP_DEF     : TREE;¶¶    NEW_TYPESET : TYPESET_TYPE := EMPTY_TYPESET;¶    NEW_DEFSET  : DEFSET_
TYPE¬ := EMPTY_DEFSET;¶  begin¶¶¬      -- FOR EACH POSSIBLE NAME TYPE¶    while¬not IS_EMPTY (TEMP_NAME_TYPESET) loop¶      POP¬(TEMP_NAME_TYPESET,¬NA
ME_TYPEINTERP);¶      NAME_STRUCT := GET_BASE_STRUCT (GET_TYPE (NAME_TYPEINTERP));¶¶¬¬    -- IF¬ACCESS TYPE, CONSIDER DESIGNATED TYPE¶      if NAME_ST
RUCT.TY = DN_ACCESS then¶        NAME_STRUCT¬:= GET_BASE_STRUCT (D (SM_DESIG_TYPE, NAME_STRUCT));¶      end¬if;¶¶¬¬    -- IF¬IT IS RECORD OR TASK TYPE
¶      if NAME_STRUCT.TY = DN_RECORD or NAME_STRUCT.TY = DN_TASK_SPEC or NAME_STRUCT.TY in¬CLASS_PRIVATE_SPEC then¶¶¬¬¬  -- GET REGION DEF¶        NAM
E_TYPE_ID := D (XD_SOURCE_NAME, NAME_STRUCT);¶        if NAME_TYPE_ID.TY = DN_TYPE_ID¬then¶¬NAME_TYPE_ID := D (SM_FIRST, NAME_TYPE_ID);¶        end if
;¶      NAME_DEF :=¬GET_DEF_FOR_ID (NAME_TYPE_ID);¶¶¬¬¬  -- SEARCH DEFLIST¬FOR COMPONENTS OR ENTRIES IN THAT REGION¶        TEMP_DEFLIST := DESIGNAT
OR_DEFLIST;¶        while not IS_EMPTY (TEMP_DEFLIST) loop¶¬POP (TEMP_DEFLIST, TEMP_DEF);¶¬if NAME_DEF = D (XD_REGION_DEF, TEMP_DEF) then¶¶¬  declare¶
¬    HEADER¬:TREE¬:= D( XD_HEADER, TEMP_DEF );¶¬  begin¶¬    if  HEADER.PT =¬HI  and then  HEADER.NOTY in CLASS_BOOLEAN¶¬    then¶¬¬¬¬¬      -- IN ERR
OR, RETURN THIS ONE AND QUIT LOOKING¶¬      NEW_DEFSET :=¬EMPTY_DEFSET;¶¬      ADD_TO_DEFSET¬(NEW_DEFSET, TEMP_DEF);¶¬      DEFSET       := NEW_DEFSET
;¶¬      NAME_TYPESET := EMPTY_TYPESET;¶¬      return;¶¬    end if;¶¬  end;¶¶¬  ADD_TO_TYPESET (NEW_TYPESET, NAME_TYPEINTERP);¶¬  ADD_TO_DEFSET (NEW_D
EFSET, TEMP_DEF, GET_EXTRAINFO (NAME_TYPEINTERP));¶¬end if;¶        end loop;¶¶¬¬¬  -- RETURN NEW SETS¶      end¬if;¶    end loop;¶    NAME_TYPESET :=
¬NEW_TYPESET;¶    DEFSET       :=¬NEW_DEFSET;¶¶  end¬FIND_SELECTED_DEFS;¶¬------------------¶¶¶end¬VIS_UTIL;¶¬--------¶
```